Eureka Discovery Server — Integration Summary

Project: order-easy

Why Do We Need Eureka?

The Problem (Before Eureka)

In a microservices architecture, services need to talk to each other. Without Eureka, you had to **hardcode URLs** in your Feign clients:

```java
// BAD — hardcoded, breaks if port/IP changes
@FeignClient(name = "inventory-service", url = "http://localhost:8082")
public interface InventoryClient { ... }
```

This causes 3 real problems:

Problem	Impact
Hardcoded `localhost`	Breaks in any non-local environment (Docker, cloud, prod)
Hardcoded port	If you run 2 instances of inventory-service, which port do you use?
No load balancing	You can't distribute traffic across multiple instances

The Solution (After Eureka)

Eureka acts as a **phone directory for your services.** Every service says *"I'm alive, I'm at this IP:port"* on startup. Anyone who needs to call it just asks Eureka for the current address.

```java
// GOOD — service name only, Eureka resolves dynamically
@FeignClient(name = "inventory-service")
public interface InventoryClient { ... }
```

What Was Built

New Module Created

```
backend/
└── discovery-server/          ← NEW
    ├── pom.xml
    └── src/main/
        ├── java/
        │   └── DiscoveryServerApplication.java
        └── resources/
            └── application.yml
```

application.yml (Discovery Server):

```yaml
```

```yaml
server:
  port: 8761

eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false   # doesn't register with itself
    fetch-registry: false         # doesn't fetch its own registry
```

Step-by-Step Integration

Step 1 — Create the Discovery Server

- Created `backend/discovery-server` as a new Spring Boot module
- Added `spring-cloud-starter-netflix-eureka-server` dependency
- Annotated main class with `@EnableEurekaServer`
- Configured as **standalone** (not registering with itself)
- Port: **8761**

Step 2 — Configure All Services as Eureka Clients

Updated all 10 services: **API Gateway, Auth, Order, Inventory, Product, Cart, Payment, Delivery, Tracking, Notification**

Dependencies added to each service's pom.xml:

```xml
```

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Load balancer added to services that make outbound calls (Gateway, Order, Cart):

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

application.yml update for each client service:

```yaml
```

```yaml
eureka:
  instance:
    instance-id: ${spring.application.name}:${server.port}
    prefer-ip-address: true      # uses actual IP, not hostname
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

management:
  endpoints:
    web:
      exposure:
        include: health, info    # Eureka reads health to mark UP/DOWN
```

Step 3 — Refactor Feign Clients

Removed hardcoded `url` attributes from all Feign clients.

Before:

```java
@FeignClient(name = "inventory-service", url = "http://localhost:8082")
public interface InventoryClient {
    @GetMapping("/api/inventory/check")
    boolean checkStock(@RequestParam Long productId);
}
```

After:

```java
```

```
@FeignClient(name = "inventory-service")   // Eureka resolves this automaticall
public interface InventoryClient {
    @GetMapping("/api/inventory/check")
    boolean checkStock(@RequestParam Long productId);
}
```

Services refactored: `InventoryClient` in **order-service**, `ProductClient` in **cart-service**

Step 4 — Refactor API Gateway Routing

Before (hardcoded ports in application.properties):

```properties
spring.cloud.gateway.routes[0].uri=http://localhost:8083
```

After (dynamic load-balanced routing):

```properties
spring.cloud.gateway.routes[0].uri=lb://order-service
spring.cloud.gateway.routes[1].uri=lb://inventory-service
spring.cloud.gateway.routes[2].uri=lb://auth-service
# ... and so on for all services
```

The `lb://` prefix tells the gateway: *"ask Eureka for the current location of this service, then load balance across all available instances."*

Step 5 — Update Orchestration Scripts

start-all.sh — Discovery server starts first, then waits:

```bash
```

```bash
# Start discovery server first
cd backend/discovery-server && ./mvnw spring-boot:run &
echo "Waiting 15 seconds for Eureka to be ready..."
sleep 15

# Now start all other services
cd backend/api-gateway && ./mvnw spring-boot:run &
cd backend/auth-service && ./mvnw spring-boot:run &
# ... all remaining services
```

stop-all.sh — Explicitly stops discovery server:

```bash
pkill -f "discovery-server"
pkill -f "api-gateway"
# ... all services
```

status-check.sh — Monitors Eureka health as foundation:

```bash
curl -s http://localhost:8761/actuator/health | grep '"status":"UP"'
# Then check each individual service
```

How to Verify It Works

1. **Start everything:**

```bash
```

```
./start-all.sh
```

2. **Open Eureka Dashboard:**

```
http://localhost:8761
```

You should see all 10 services listed with status **UP**

3. **Run status check:**

```bash
./status-check.sh
```

4. **Check Gateway logs** to see discovery-based routing:

```bash
tail -f /tmp/api-gateway.log
```

What This Unlocks

Feature	How
Zero hardcoded URLs	All communication uses service names
Multiple instances	Run 2+ instances of any service — gateway auto load balances
Self-healing	If a service crashes, Eureka marks it DOWN; traffic stops going to it

Feature	How
Environment agnostic	Works locally, in Docker, on cloud without config changes
Production-grade IDs	`prefer-ip-address: true` + unique `instance-id` per instance

Architecture Principle

> Eureka solves the **service location problem** — the question of *"where exactly is service X running right now?"*
> Your code only needs to know the **name** of a service. Eureka handles the rest.

```
Before: order-service → http://localhost:8082/api/inventory  (fragile)
After:  order-service → inventory-service → Eureka resolves → actual IP:port
(resilient)
```